AutoLandry团队章程与 AI 工具选型报告

项目名称： 校园互助服务平台 (CampusHub)

所属课程： 软件工程与计算 II

团队成员： 曾品涵，田翔宇，史益瑄，王佳宝

日期： 2026年4月15日

一、 团队角色分工与职责阐述

本团队根据项目需求，设立四个核心角色。各成员需对所承担角色的职责有深刻理解，并对核心目标负责。

1.1 需求负责人

承担成员： 史益瑄

角色理解： 需求负责人是用户与开发团队之间的桥梁，负责将模糊的校园互助场景转化为清晰的软件需求。

核心职责与目标：

负责 P1 阶段的用户访谈、问卷设计及调研分析。

撰写并维护《需求规格说明书》（基于 IEEE 830 标准）。

执行用例建模，编写至少 15 条用户故事，并使用 MoSCoW 方法进行优先级排序。

核心目标： 确保系统功能覆盖 MVP 范围内的所有 P0 级功能，并保证需求的可验证性。

1.2 架构负责人

承担成员： 王佳宝

角色理解： 架构负责人负责系统的“骨架”设计，平衡技术先进性与开发成本，确保系统在处理校园高并发时的稳定性。

核心职责与目标：

主导 P2 阶段的技术选型与系统架构设计。

撰写架构决策记录 (ADR)，明确前后端分离策略及数据库选型。

设计系统 API 规范及数据库 ER 图（P3 阶段）。

核心目标： 建立高可用、可扩展的技术架构，满足性能指标。

1.3 开发负责人

承担成员： 田翔宇

角色理解： 开发负责人是工程质量的把关人，负责制定开发规范，并在 AI 辅助编程的环境下确保代码的健壮性和逻辑正确性。

核心职责与目标：

建立团队编码规范，并负责核心业务模块的开发。

执行代码审查 (Code Review)，确保 AI 生成的代码符合质量要求。

负责项目目录结构初始化及基础环境配置。

核心目标： 实现 CampusHub 的最小可行产品 (MVP)，确保 P0 功能 100% 交付。

1.4 测试负责人

承担成员： 曾品涵

角色理解： 测试负责人负责建立全生命周期的质量保障体系，不仅关注功能实现，更关注系统的安全性与健壮性。

核心职责与目标：

制定测试策略，设计涵盖正常流与异常流的测试用例。

利用 AI 测试工具辅助生成单元测试，确保护航关键路径。

执行集成测试与回归测试，撰写最终的质量报告。

核心目标： 确保核心模块测试覆盖率达标，防止 SQL 注入与 XSS 等安全漏洞。

---

2. AI 工具链选型报告

2.1 AI 编程助手 (AI Coding Assistant)

主要用于 P4 核心编码阶段，目标是加速 CRUD 代码生成和复杂逻辑实现。

候选工具	优势分析	潜在局限	评估结论
GitHub Copilot	历史最悠久，IDE 兼容性极佳，插件生态成熟 。	补全较为碎片化，缺乏对整个项目上下文的深度理解。	备选
Cursor	基于 Claude 3.5 Sonnet，具备“全库索引”和“对话修改代码”能力 。	需要改变传统的编码习惯，依赖稳定的网络环境。	**最终选择**
通义灵码 (Lingma)	对中文注释和国产 IDE 支持极佳，国内访问速度快 。	在处理极复杂的算法逻辑时，推理能力略逊于海外顶级模型。	备选

> **选择理由：** 考虑到本项目涉及多个功能模块（用户、订单、评价等），**Cursor** 的 Composer 模式能跨文件生成代码，非常适合快速构建项目原型和状态机逻辑 。

2.2 AI 对话与文档工具 (AI Chat & Docs)

用于 P1 需求分析、文档撰写及 P2 架构设计决策 。

候选工具	核心能力描述	适用场景	评估结论
ChatGPT (GPT-4o)	逻辑极其严密，遵守指令的能力最强。	编写 IEEE 830 标准的需求文档初稿 。	备选
Gemini3	文风更自然，代码理解力顶尖，能生成更具可读性的文档。	编写用户故事和复杂的用例描述 。	**最终选择**
DeepSeek-V3	国内顶尖模型，数学与代码逻辑极强，API 成本极低。	辅助架构决策讨论和技术选型对比。	备选

> **选择理由：** **Gemini3**在理解复杂的工程约束（如非功能性需求）时表现更“聪明”，能有效减少人工修改的幅度。

2.3 AI 代码审查工具 (AI Code Review)

确保 AI 生成的代码不会引入安全漏洞（如 SQL 注入）并符合规范。

候选工具	功能亮点	团队关注点	评估结论
CodeRabbit	自动总结 PR 变更，逐行提供改进建议，支持多种语言 。	能否识别出 AI 捏造的冗余函数?	**最终选择**
GitHub PR-Agent	开源且集成简单，支持快捷指令（如 /review, /describe）。	审查深度相对 CodeRabbit 稍浅。	备选
SonarQube AI	传统的静态扫描结合 AI 解释，专业性极高。	配置较重，不适合 10 周的快速开发节奏。	不选择

> **选择理由：** **CodeRabbit** 的反馈最即时，能作为《AI 协作契约》中“人工审查”的有效前哨，拦截明显的低级错误 。

2.4 AI 测试工具 (AI Testing Tools)

用于生成单元测试、集成测试及模拟用户交互场景。

候选工具	自动化程度	覆盖范围	评估结论
CodiumAI	根据业务代码自动生成全面的测试套件（Test Suite） 。	单元测试、API 测试。	**最终选择**
Diffblue Cover	针对 Java (Spring Boot) 的自动化测试生成能力极强。	如果技术栈选 Java，此项为首选。	备选
Playwright AI	利用 AI 自动修复因 UI 变动导致的端到端（E2E）测试失败。	适合后期 UI 频繁变动的回归测试。	备选

> **选择理由： CodiumAI** 提供的测试计划（Test Plan）能帮助测试负责人发现 AI 生成代码中的边界异常流。